

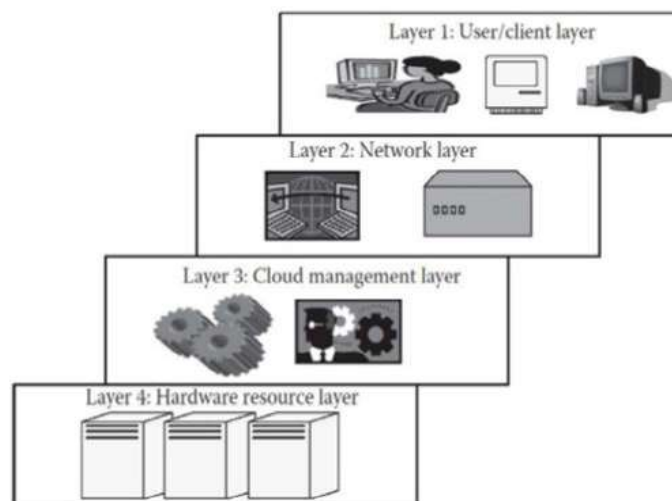
## UNIT-III

**Cloud computing Architecture and Management:** Managing the cloud and managing the cloud infrastructure using AWS cloud Front, Managing the cloud application, Managing Identity and Access (IAM), Migrating Application to cloud, Phases of cloud migration, Approaches for Cloud Migration.

### Cloud computing Architecture and Management

Cloud computing is the foundational technology that allows organizations of all sizes to store information and access services globally via the internet. To manage these services effectively, we must look at the **Architecture** (the functional hierarchy) and the **Anatomy** (the structural components).

### Cloud Architecture



Architecture represents the **hierarchical view** of the cloud. It describes the technical ecosystem by mapping out the components that the cloud is built upon, as well as the services that depend on that technology to function.

In simple terms, Architecture is the "Roadmap." it explains how a user's request travels from a device, through the internet, into a management system, and finally to the physical hardware.

### Layer 1 (User/Client Layer)

The **User/Client Layer** acts as the primary entry point of cloud architecture, representing the hardware and software interface where all user interactions are initiated. Whether through a mobile phone, a handheld device, or a desktop computer, this layer provides the necessary

gateway for users to access cloud-hosted applications. While these applications often look and feel like standard websites to the end-user, their internal mechanics are far more complex, as the actual processing and data storage are distributed across remote servers rather than residing solely on the local machine.

Within this layer, devices are categorized as either **Thin** or **Thick clients** based on their local processing capabilities. A **Thin Client** refers to a device or application, such as a basic mobile device or software like **Office 365** and **Microsoft Outlook**, that possesses very low local processing power. These clients are almost entirely dependent on a remote system to function, acting primarily as a display terminal that sends user inputs to the cloud for execution. Because they have minimal storage and computational strength, they require a constant connection to the cloud to remain operational.

In contrast, **Thick Clients** are general-purpose computers, such as **Windows PCs** or **macOS laptops**, that possess adequate internal hardware to perform complex tasks independently. These systems have sufficient capability for standalone processing, meaning they can run heavy software locally and only connect to the cloud to sync data or access specialized remote services. Ultimately, this layer is defined by these diverse client devices, each serving as the critical starting point for the entire cloud computing experience by translating human actions into digital requests.

---

## **Layer 2: The Network Layer**

The **Network Layer** serves as the essential "bridge" that allows user devices to connect to the cloud infrastructure. This connectivity is the lifeline of the entire cloud model, as it is the medium through which all services are delivered to the customer. While a public cloud typically exists in a remote, abstract location that is hidden from the user, it remains accessible globally through this layer. In contrast, a private cloud may utilize a **Local Area Network (LAN)** for its connectivity, but in both cases, the cloud's functionality is entirely dependent on the stability and speed of the network being used.

To ensure a smooth experience, cloud providers often define a **minimum bandwidth** requirement for users to access their public or private resources. This ensures that the data can flow back and forth without significant delays. However, it is important to note that this layer

typically falls outside the scope of **Service Level Agreements (SLAs)**. This means that while a provider guarantees the performance of their servers, they do not take responsibility for the quality of the internet connection between the user's home or office and the cloud data center.

Ultimately, the Network Layer is about accessibility and the "Highway" of data. Whether the user is connecting via a 5G mobile network, a home fiber connection, or a corporate LAN, this layer must be robust enough to handle the traffic. Because the cloud is a recent technology that is completely dependent on the internet for its functioning, any failure in this layer results in a total loss of access to cloud services, regardless of how powerful the underlying hardware may be.

---

### **Layer 3: The Cloud Management Layer**

The **Cloud Management Layer** acts as the "brain" of the cloud architecture, consisting of specialized software that manages the relationship between the user and the physical hardware. This layer includes cloud operating systems and management software that provide an interface for the user to interact with the data center. Its primary responsibilities include **resource management**, such as scheduling and provisioning tasks, and **optimization**, which involves consolidating servers and storage workloads to ensure the system runs at maximum efficiency.

Unlike the Network Layer, the Cloud Management Layer is strictly governed by **Service Level Agreements (SLAs)**. This means that the efficiency of operations at this level directly affects the legal and financial agreements between the provider and the user. Any delay in processing a request or a discrepancy in providing a service can lead to an **SLA violation**. In such cases, the service provider is usually required to pay a penalty to the user, making this layer one of the most commercially sensitive parts of the architecture.

Popular software platforms at this level vary depending on the type of cloud being managed. For public clouds, massive systems managed by providers like **Amazon Web Services (AWS)** or **Microsoft Azure** dominate the landscape. For private clouds, open-source software like **OpenStack** or **Eucalyptus** allows organizations to create, deploy, and manage their own internal cloud environments. Because this layer controls how resources are divided and governed, it is often referred to as the true "infrastructure" of the cloud.

---

### **Layer 4: The Hardware Resource Layer**

The **Hardware Resource Layer** is the physical foundation of the cloud, consisting of the actual machines and equipment located in a back-end data center. In a public cloud, this is a massive facility filled with interconnected hardware resources, while in a private cloud, it may be a smaller collection of high-configuration systems. This layer is where the "heavy lifting" happens, providing the raw power needed to run applications and store vast amounts of data for millions of users simultaneously.

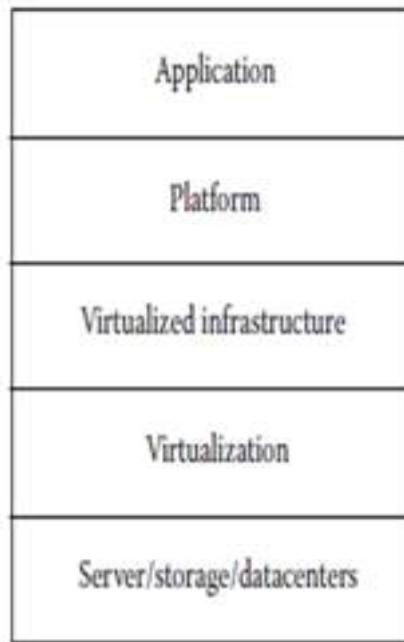
This layer is arguably the most critical for governing **SLAs**, particularly regarding availability and speed. Whenever a user initiates a request, the hardware must be able to provision the required resources as quickly as possible within the time limits defined by the agreement. Because any hardware failure or slow performance can trigger a penalty for the service provider, data centers are equipped with high-speed internal networks and highly efficient algorithms to ensure data moves from the storage units to the management layer instantly.

Furthermore, the scale of this layer is often immense, with a single cloud potentially spanning multiple data centers across different geographic regions. Conversely, several different cloud environments can share the same physical data center through virtualization. To maintain high standards, these facilities are designed with extreme redundancy in power, cooling, and connectivity, ensuring that the hardware remains operational 24/7 to meet the demanding requirements of modern global enterprises.

---

### **Anatomy of the Cloud**

- Cloud anatomy can be simply defined as the structure of the cloud. Cloud anatomy cannot be considered the same as cloud architecture. It may not include any dependency on which or over which the technology works , whereas architecture wholly defines and describes the technology over which it is working.
- Architecture is a hierarchical structural view that defines the technology as well as the technology over which it is dependent or/and the technology that are dependent on it. Thus, anatomy can be considered as a part of architecture.



There are basically **five components** of the cloud:

1. **Application:** The upper layer is the application layer. In this layer, any applications are executed.
  2. **Platform:** This component consists of platforms that are responsible for the execution of the application. This platform is between the infrastructure and the application.
  3. **Infrastructure:** The infrastructure consists of resources over which the other components work. This provides computational capability to the user.
  4. **Virtualization:** Virtualization is the process of making logical components of resources over the existing physical resources. The logical components are isolated and independent, which form the infrastructure.
  5. **Physical hardware:** The physical hardware is provided by server and storage units.
- 

## Managing the Cloud

Cloud Management is the essential practice of exercising control and oversight over an organization's digital infrastructure, services, and applications within a cloud environment. As businesses increasingly migrate their IT setups to the cloud, they do so to capitalize on the inherent flexibility, massive scalability, and cost efficiencies that modern cloud technology offers.

However, simply moving to the cloud is not enough; these resources must be systematically configured and governed to ensure they operate at peak performance. Cloud management provides the necessary framework of policies, strategies, and specialized technologies required to maintain and secure private, public, and hybrid cloud environments.

## **Why Manage the Cloud?**

The primary reason for robust cloud management is to prevent the chaos of unmonitored growth, often referred to as "cloud sprawl." Without a structured management approach, cloud resources can quickly become unorganized, leading to significant security vulnerabilities and wasted financial expenditure. Systematic management ensures that every virtual server, database, and application is tuned for optimum efficiency and strict cost control. By implementing automated governance, organizations can ensure that their cloud usage remains compliant with industry regulations while maintaining the high-speed agility that cloud computing promises.

## **Benefits of cloud management**

- Cloud management gives organizations a single point of control over the vast resources they deploy on the cloud.
- Organizations can balance innovative cloud expansion with governance, cost control, and flexibility.
- We share several benefits that organizations will gain from using cloud management solutions.

## **Understanding Cloud Management**

Cloud management is the "Mission Control" for an organization's digital assets. It involves a mix of specialized software and human-led strategies to ensure that every virtual machine, database, and application is doing exactly what it should—without breaking the bank or leaking data.

### **1. Ease of Use & Operational Efficiency**

In the "old days," adding server capacity meant ordering hardware, waiting for shipping, and manual racking. Cloud management replaces this with Software-Defined Infrastructure.

- **Self-Service Portals:** Instead of filing a ticket and waiting days, developers can spin up the resources they need via a user-friendly dashboard.
- **Infrastructure as Code (IaC):** This allows teams to manage their setup using configuration files, making it easy to replicate environments across different regions instantly.

- **Unified Visibility:** Administrators get a "single pane of glass" view, meaning they don't have to toggle between five different tabs to see if their systems are healthy.

## 2. Centralized Governance & Security

As companies grow, "Shadow IT" (employees using unauthorized cloud services) becomes a risk. Centralized management acts as the digital guardrails.

- **Role-Based Access Control (RBAC):** This ensures that only authorized personnel can access sensitive data. For example, a junior dev might be able to view a database, but only a senior admin can delete it.
- **Compliance Automation:** Tools can automatically scan for violations—like an unencrypted bucket of data—and fix them or alert the team before a regulator finds out.
- **Standardization:** It ensures every department uses the same approved "blueprints" for their setups, reducing the risk of human error.

## 3. Cost & Capacity Control (Avoiding the "Bill Shock")

Cloud costs are variable, which is great for scaling but dangerous if left unmonitored. This is often referred to under the umbrella of **FinOps**.

- **Curbing "Cloud Sprawl":** It's common for a developer to start a project, finish it, and forget to turn off the expensive virtual machines. Management tools identify these "zombie resources."
- **Right-Sizing:** The system analyzes if you are paying for a "Large" server when your traffic only requires a "Small" one, then suggests an immediate downgrade to save money.
- **Predictive Analytics:** Advanced platforms use AI to forecast your next month's bill based on current usage trends, preventing nasty surprises.

## 4. Automated Incident Response & Resilience

In a cloud environment, manual intervention is too slow. Automation is the key to maintaining "High Availability."

- **Self-Healing Systems:** If a server fails, the management platform can automatically spin up a replacement and redirect traffic without the end-user ever noticing.
  - **Automated Backups:** It ensures that data is backed up across different geographic zones, so if a physical data center is hit by a natural disaster, your business stays online.
  - **Orchestration:** During a traffic spike (like a Black Friday sale), the system "auto-scales," adding more power automatically and then scaling back down when the rush is over.
-

## 1. What is Amazon CloudFront?

CloudFront is a **Content Delivery Network (CDN)** integrated into the AWS ecosystem. It acts as a middleman between your central server (the Origin) and your users.

- **Global Presence:** It uses a massive network of **Edge Locations** (data centers specifically for caching) to bring content physically closer to users.
- **Static & Dynamic Content:** While it's famous for caching "static" files like images and CSS, it also speeds up "dynamic" content (like live API calls) by using the optimized AWS private network backbone instead of the crowded public internet.

## 2. Why Use a CDN? (The Need for Speed)

The primary enemy of a good user experience is **Latency**—the time it takes for a data packet to travel from point A to point B.

### Key Benefits

- **Reduced Latency:** Shorter physical distance = faster loading.
- **Bandwidth Cost Savings:** AWS charges less for data moving out of CloudFront than for data moving directly out of an S3 bucket or EC2 server.
- **Availability:** Even if your main server goes down briefly, CloudFront can often still serve cached versions of your site.
- **Security:** CloudFront integrates with **AWS WAF** (Web Application Firewall) and **Shield** to protect your site from DDoS attacks at the "edge" before they ever reach your server.

## 3. The Mechanics: Hits vs. Misses

Understanding how CloudFront handles a request is vital for optimizing performance. This is measured by the **Cache Hit Ratio**.

| Term                   | What Happens                                                             | Result                                                                                                                      |
|------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>CloudFront Hit</b>  | The requested file is already stored in the local Edge Location's cache. | <b>Instant delivery.</b> The request never touches your origin server.                                                      |
| <b>CloudFront Miss</b> | The file is missing or has expired (TTL) in the Edge Location.           | <b>Latency.</b> CloudFront must fetch the file from the Origin, deliver it to the user, and then save a copy for next time. |



**Pro-Tip:** You can control how long a file stays in the cache using a setting called **TTL (Time To Live)**. If your content updates frequently, you set a low TTL; if it's a logo that never changes, you set a high TTL.

#### 4. How CloudFront Delivers Content (Step-by-Step)

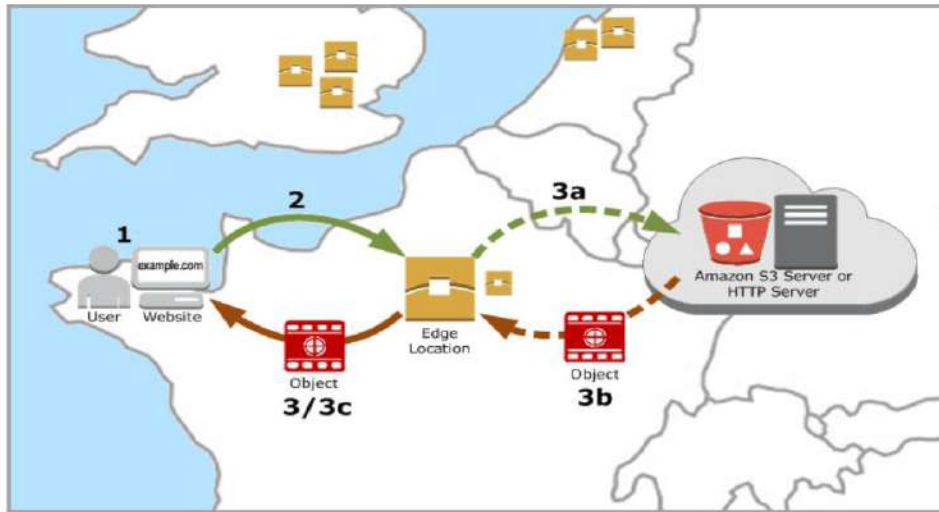
When a user clicks a link to your site, a complex but lightning-fast "handshake" occurs:

1. **The Request:** The user's browser asks for `www.example.com/logo.png`.
2. **DNS Routing:** The Domain Name System (DNS) realizes this is a CloudFront URL and routes the request to the **Point of Presence (PoP)** with the lowest latency (usually the one geographically closest to the user).
3. **The Cache Check:** \* **Scenario A (Hit):** CloudFront finds the file. It sends it back immediately. Done!
  - **Scenario B (Miss):** CloudFront sees the file isn't there.
4. **Origin Fetch (On Miss):** CloudFront sends a request to your **Origin** (e.g., an S3 Bucket or an EC2 Server).
5. **Streaming & Caching:** As the Origin sends the file back, CloudFront does two things at once:
  - It passes the data through to the user immediately (it doesn't wait for the whole file to finish downloading).
  - It saves a copy in its cache for the next user who asks for it.

#### Example

Imagine you are ordering a book.

- **Without CDN:** You order from a warehouse in another country. It takes 2 weeks to arrive.
- **With CDN (CloudFront):** The book company has a local "edge" bookstore in your city. If the book is on the shelf (**Hit**), you get it in 10 minutes. If it's not (**Miss**), the store orders it for you, and once it arrives, they keep a copy on the shelf for the next customer.



How Cloud Front delivers content to the user?

## Demo of CloudFront

### CloudFront Distribution demo steps

Creating a CloudFront distribution that pulls from an S3 bucket with **Static Website Hosting** enabled is a classic way to learn the difference between "Direct S3 Access" and "Website Endpoint Access."

Here are the step-by-step notes for your demo.

### Phase 1: The S3 Setup

#### 1. Create Bucket with Website Enabled

- **Create Bucket:** Name it (e.g., my-demo-site-2026). Keep settings default for now, but **uncheck** "Block all public access" since static website hosting requires the bucket to be public (or used with a secret Referer header).
- **Enable Hosting:** Go to **Properties > Static website hosting > Edit**.
  - Select **Enable**.
  - Index document: index.html.
  - **Save changes.** Note the "Bucket website endpoint" URL provided at the bottom (e.g., http://...s3-website...).

## 2. Upload Sample Files

- Upload a basic index.html file:

HTML

```
<h1>Hello from S3 Website!</h1>
```

- **Set Permissions:** In the **Permissions** tab, add a Bucket Policy to allow public s3:GetObject so the website can actually serve the files.

## 3. Test Website

- Open the **Bucket website endpoint** URL in your browser. If you see "Hello from S3 Website!", Phase 1 is a success.

### Phase 2: The CloudFront Setup

## 4. Create CloudFront Distribution

- Origin Domain: Crucial Step! \* *Do not* select the bucket from the dropdown.
  - Instead, paste the S3 Website Endpoint URL (the one without https://) into the box. This tells CloudFront to treat S3 like a standard web server.
- Viewer Protocol Policy: Select Redirect HTTP to HTTPS.
- Default Root Object: Set to index.html.
- Settings: Keep everything else as default and click Create Distribution.

## 5. Test the Distribution

- Wait for the "Last Modified" status to change from *Deploying* to a timestamp (usually 3–5 minutes).
- Copy the **Distribution Domain Name** (e.g., d123.cloudfront.net).
- Paste it into your browser. You should see your site, now served via HTTPS and global edge locations.

### Phase 3: The Cleanup

6. Disable and Delete—the service what you spined up

---

## Identity and Access Management (IAM)

### What is IAM?

IAM is a global service that lets you securely control access to AWS resources. It provides the framework to ensure that the right people and systems have the right access to the right resources under the right conditions.

### Key Concepts:

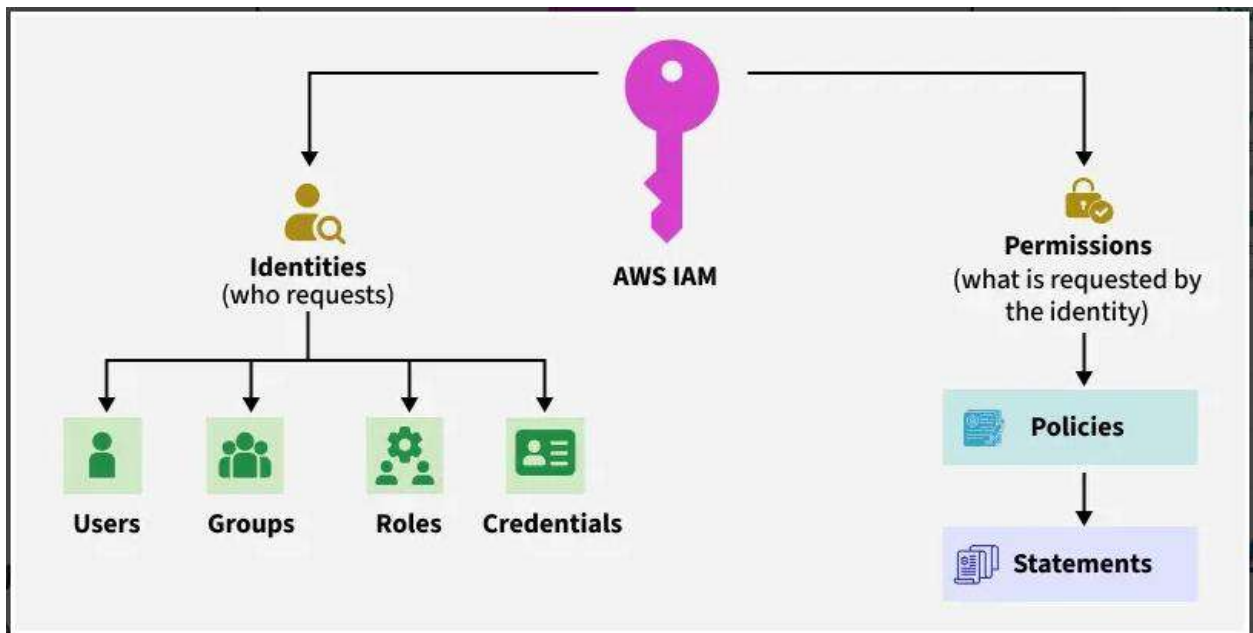
- **Authentication:** Confirming identity (Username/Password, Access Keys).
- **Authorization:** Granting permission (What can this user actually click or change?).
- **Global Service:** Unlike EC2 or S3, IAM is **not region-specific**. When you create a user, they exist globally across all AWS regions.

### IAM: The Core of AWS Security

IAM (Identity and Access Management) is a **Global Service**. It does not require you to select a region because a user created in your account can access resources anywhere in the world (if permitted).

### The Root User vs. IAM Users

- **Root User:** The "Super Admin" created with your email address. It has unrestricted access to everything, including billing and account closing.
  - **Rule for Students:** Never use the Root User for daily tasks. Create it, enable MFA, and "lock it away."
- **IAM User:** A specific identity created for a person or an application.
  - **New User Default:** Every new IAM user starts with **Zero Permissions** (Implicit Deny). You must explicitly grant them access.



## Why IAM?

The **Root User** is created when you first open an AWS account. It has "God Mode" powers—it can delete the entire account and access billing.

- **Best Practice:** Lock away the Root User credentials and use IAM users for daily tasks.
- **Control:** Without IAM, you cannot limit a junior developer from accidentally deleting a production database.
- **Cost Management:** IAM allows you to track which user is launching expensive resources under a single master bill.

## AWS IAM Features

- **Centralized control :** Root User can control creation and cancellation of each user's security credentials. Root user can also control what data in the aws system users can access and how they can access.
- **Shared Access :** Users can share the resources for the collaborative projects.
- **Granular permissions:** It is used to set a permission that user can use a particular service but not other services.
- **Free to use:** AWS IAM is a feature of AWS account which is offered at no additional charge. You will be charged only when you access other AWS services by using IAM user.
- **Multifactor Authentication:** An AWS provides multifactor authentication as we need to enter the username, password, and security check code to log in to the AWS Management Console.

## Advantages of IAM

- Centralized control of your AWS Account

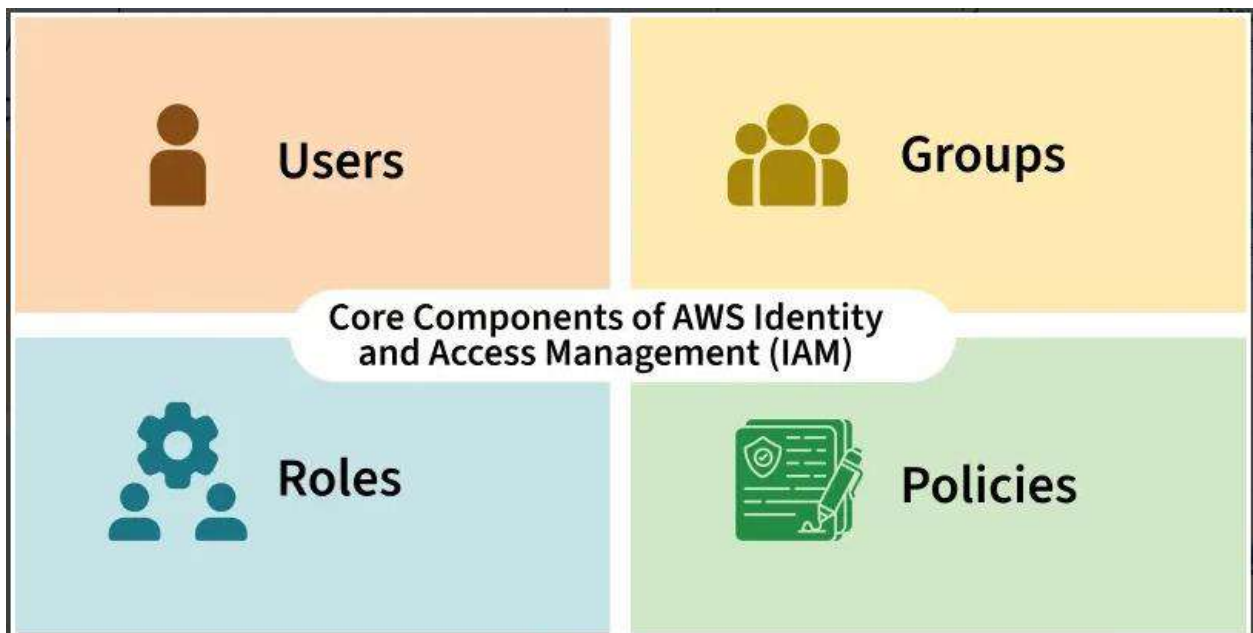
- Shared access to your AWS Account
- Granular permissions
- Identify federation ( users can login using LinkedIn , Facebook etc)
- Multifactor Authentication ( password and OTP )
- Setup password rotation policy ( eg: password will expire every 30 days )

### Important Points

- IAM is universal . It does not apply to regions at this time.
- The “root account” is simply the account created when first setup your AWS account . It has complete Admin access.
- New Users are assigned Access Key ID & Secret Access Keys when first created.
- These are not same as a password and you cannot use the Access Key ID & Secret Access Key to join in to the console . You can use this to access Command line.
- You can create and customize your own password rotation.

### 3. The Four Core Components

- Think of these as the "Building Blocks" of AWS security.



- **USERS**- End Users ( people )
- **GROUPS**– Collections of users, under one set of permissions.
- **ROLES** – We can create roles and assign them to AWS resources
- **POLICIES**– Set of permissions

#### A. IAM Users (The "Identity")

An IAM User represents a **person** or **application** that interacts with AWS. It is a persistent identity.

- **Two Ways to Access:**

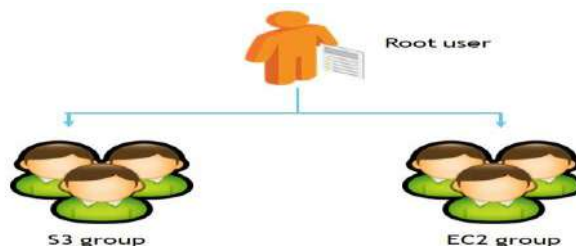
1. **Console Access:** A username and password used to log into the AWS web interface. Students should always enable **MFA (Multi-Factor Authentication)** here.
  2. **Programmatic Access:** Uses an **Access Key ID** and **Secret Access Key**. This is used for the AWS CLI (Command Line), SDKs, or APIs.
- **Key Concept:** By default, a new user has **no permissions**. They can log in but cannot see or touch anything until they are given a policy or put into a group.



## B. IAM Groups (The "Organizer")

An IAM Group is a collection of IAM Users. You do not "log in" to a group; it is simply a way to manage permissions at scale.

- **Inheritance:** Any policy attached to a group is automatically "inherited" by every user inside that group.
- **Best Practice:** Instead of attaching policies to individual users, students should create groups (e.g., Admins, Developers, Auditors) and move users in and out of them.
- **Important Limitation:**
  - Groups cannot be nested (you can't put a group inside a group).
  - Groups are not "identities"—they cannot be identified as the "Principal" in a policy.



## C. IAM Roles (The "Hat")

An **IAM Role** is an identity that you can create in your account that has specific permissions. It is similar to an IAM user, but with one major difference: **it does not have permanent credentials (password or access keys) attached to it**.

- **How it works:** Instead of being uniquely associated with one person, a role is intended to be **assumed** by anyone (or anything) that needs it.

- **The Analogy:** If an IAM User is a permanent employee badge, an IAM Role is a **keycard left in a lockbox**. Anyone with the code can take the card, use it to open the door, and must return it (or it expires) afterward.
- **Why use Roles?**
  - **No Credentials to Leak:** Since there are no Access Keys stored on the EC2 instance, if a hacker breaks into your server, they won't find any permanent keys to steal.
  - **Service-to-Service Access:** It allows an AWS service (like EC2) to talk to another (like S3) natively and securely.

### Procedure: Attaching a Role to an EC2 Instance

This procedure allows an EC2 instance to perform S3 actions without the student ever running aws configure.

1. **Create the Role:** Go to IAM > Roles > Create role.
2. **Select Trusted Entity:** Choose AWS Service. Under the "Service or use case" dropdown, select EC2. (This tells AWS: "I trust the EC2 service to use this role").
3. **Add Permissions:** Search for and check AmazonS3FullAccess.
4. **Name & Review:** Name it (e.g., EC2-S3-ReadOnly-Role) and click **Create role**.
5. **Attach to Instance:**
  - Go to the **EC2 Dashboard**.
  - Select your running instance.
  - Click **Actions > Security > Modify IAM role**.
  - Select the role you just created and click **Update IAM role**.
6. **Verify via CLI:**
  - SSH/Connect into your EC2 instance.
  - Type `aws s3 ls`.
  - **The Result:** The command works immediately. You did **not** have to provide an Access Key or Secret Key. AWS automatically provided **Temporary Security Credentials** to the instance.



### D. IAM Policies (The "Rulebook")



## 1. What is a Policy?

A **Policy** is an object in AWS that, when associated with an identity (User, Group, Role) or resource, defines their permissions. A Policy is a **JSON document** that defines permissions. It is the logic that says "Yes" or "No."

- **JSON Format:** AWS stores these as JavaScript Object Notation (JSON) documents.
- **Evaluation:** Every time a request is made (via Console, CLI, or SDK), AWS looks at all applicable policies to see if there is an **Allow**.

## 2. The 3 Main Elements of a Policy

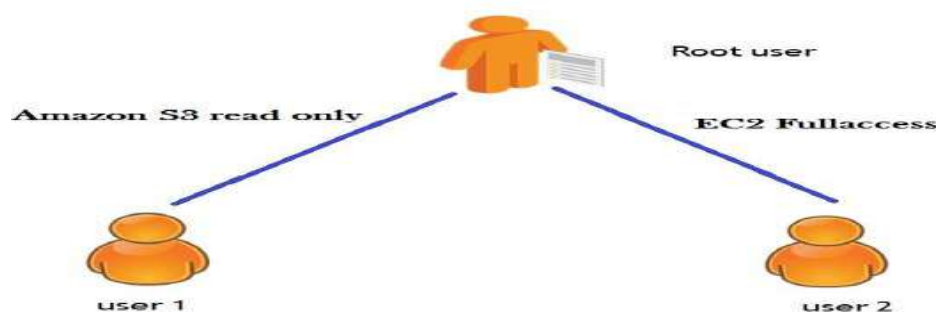
Students must be able to identify these three "blocks" in a JSON document:

- **Effect:** Is the result an Allow or a Deny?
- **Action:** The specific API call (e.g., `ec2:RunInstances`, `s3:ListBucket`).
- **Resource:** The specific AWS resource the action applies to (e.g., a specific S3 bucket ARN).

## 3. Policy Types (Theory Reference)

While there are many types, students should focus on the top two:

1. **Identity-based Policies:** Attached to Users, Groups, or Roles. (e.g., "This User can access S3").
2. **Resource-based Policies:** Attached directly to a resource like an S3 Bucket. (e.g., "This Bucket allows access from User X").



## 4. Access Method

### A: The Management Console (GUI)

The GUI is designed for human interaction via a web browser. It is best for manual configuration and visual monitoring.

- **Credentials:** Requires a Username and Password.
- **Sign-In Flow:** IAM users must use a specific sign-in URL containing the AWS Account ID or alias.

- **Security:** Access is protected by MFA (Virtual or Hardware tokens).
- **Theory of Least Privilege:** A user logged into the console can only "see" services they have explicit "Allow" permissions for. Other dashboards will display "Access Denied" or "Unauthorized" errors.

#### 4. Access Method for IAM Users

##### B: The Command Line Interface (CLI)

The CLI allows for Programmatic Access, enabling developers and system administrators to manage AWS through scripts and automation.

##### The Security Logic

Unlike the GUI, the CLI does not use passwords. It uses **Access Keys**:

1. **Access Key ID:** (Public) Identifies the user.
2. **Secret Access Key:** (Private) Used to digitally sign requests.

**Note:** This key is only shown once during creation.

##### Theoretical Configuration (aws configure)

To connect a standalone system to AWS, four parameters must be defined locally:

- **Identity:** Access Key ID & Secret Access Key.
- **Region:** The default physical location for resources (e.g., us-east-1).
- **Format:** The data structure of the response (typically json).

##### Command Syntax Theory

Every CLI command follows a specific hierarchy to ensure the request is routed correctly:

`$$ {aws} {service} {operation} {parameters}`

*Example:* `aws s3 mb s3://my-new-bucket`

## Procedure: IAM Access via GUI & CLI

### Part A: GUI Access (Management Console)

**Objective:** Create an IAM user with restricted permissions and verify access via the AWS web portal.

#### 1. Administrative Setup (Root User)

- **Step 1:** Log in to the AWS Management Console as the **Root User**.
- **Step 2:** Search for **IAM** under the "Security, Identity, & Compliance" category.
- **Step 3:** Navigate to Users and click Create user.

- **Step 4:** User Details: Enter a name (e.g., S3\_Specialist). Check the box for "Provide user access to the AWS Management Console." – custom password.
- **Step 5:** Set Permissions: Select "Attach policies directly." Search for and select AmazonS3FullAccess.
- **Step 6:** Review & Create: Complete the wizard. On the final page, click Download .csv file.

**Note:** This file contains your Console Password and the unique Sign-in URL.

- **Step 7:** Copy your 12-digit Account ID (found in the top-right dropdown or the .csv) and Sign Out.

## 2. Verification (IAM User Login)

- **Step 8:** Use the Sign-in URL from the .csv (or go to the AWS sign-in page and select **IAM User**).
- **Step 9:** Enter the **Account ID**, then the **IAM Username** and **Password** from the .csv.
- **Step 10: The Logic Test:**
  - **Test 1 (Failure):** Navigate to **EC2**. Try to view instances. You will see "API Error" or "Access Denied." This proves the user is restricted.
  - **Test 2 (Success):** Navigate to **S3**. Create a new bucket. It will work perfectly because of the attached policy.

---

## Part B: CLI Access (Programmatic Access)

**Objective:** Access AWS resources from a standalone system (CMD/Terminal) using security keys.

### 1. Credential Generation

- **Step 1:** While logged in as an Admin/Root, go to **IAM > Users** and click on your user.
- **Step 2:** Go to the **Security credentials** tab. Scroll to **Access keys** and click **Create access key**.
- **Step 3:** Select **Command Line Interface (CLI)**. Acknowledge the warning and click **Next**.
- **Step 4: Download the .csv file.** > **Critical:** This contains the **Access Key ID** and **Secret Access Key**. If lost, you must delete and recreate them.

### 2. Standalone System Configuration

- **Step 5:** Install the **AWS CLI Tool** from google on your computer.
- **Step 6:** Open your Command Prompt (CMD) or Terminal.
- **Step 7:** Type aws configure and press Enter.
- **Step 8:** Provide the following from your .csv:

- **Access Key ID**
- **Secret Access Key**
- **Default region:** (e.g., ap-south-1)
- **Default output:** json

### 3. Verification via Commands

- **Step 9:** Run `aws s3 ls`. This should list your buckets.
- **Step 10:** Run `aws s3 mb s3://your-unique-bucket-name`. (Success)
- **Step 11:** Run `aws iam list-users`.
  - **Result:** You will get an (**AccessDenied**) error.

This confirms that the CLI is respecting the same IAM policy as the GUI.

## IAM User Groups

### 1. Why use User Groups?

In a large organization, attaching policies to every individual user is inefficient. **IAM User Groups** allow you to:

- **Centralize Management:** Assign permissions to the group once.
- **Inheritance:** Every user added to the group automatically "inherits" those permissions.
- **Easy Offboarding:** When an employee leaves a department, simply remove them from the group to revoke their access.

### 2. Procedure: Creating a Group and Adding Users

#### Phase 1: Creating the User Group (Root/Admin Task)

- Step 1: Log in to the AWS Console and navigate to IAM.
- Step 2: Click User groups in the left sidebar and select Create group.
- Step 3: Group Name: Enter a descriptive name (e.g., S3-Admins-Group).
- Step 4: Attach Permissions: Under "Attach permissions policies," search for AmazonS3FullAccess and check the box.
- Step 5: Click Create group.

#### Phase 2: Creating a User within the Group

- Step 6: Navigate to Users -> Create user.
- Step 7: Enter the User name and enable Console Access (for GUI) or Access Keys (for CLI).
- Step 8: Set Permissions: Instead of "Attach policy directly," select "Add user to group."

- Step 9: Select the S3-Admins-Group you created in Phase 1.
  - Step 10: Click Next and Create user.
- 

### 3. Verification: GUI and CLI Access

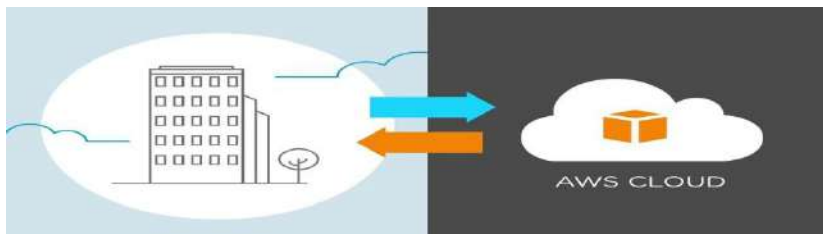
#### GUI (Console) Access

- Step 11: Log in with the new IAM User credentials.
- Step 12: Navigate to S3. You will have full access to create/delete buckets because you are a member of the group.
- Step 13: Navigate to VPC or EC2. You will get Access Denied.

#### CLI (Programmatic) Access from an EC2 Instance

- Step 14: Open the terminal of your standalone system or EC2 instance.
- Step 15: Run aws configure and enter the Access Keys generated for the user.
- Step 16: Run aws s3 ls.

### Migrating Application to Cloud



- Cloud migration encompasses moving one or more enterprise applications and their IT environments from the traditional hosting type to the cloud environment, either public, private, or hybrid.
- Cloud migration presents an opportunity to significantly reduce costs incurred on applications.
- This activity comprises of different strategies like 6 R's and phases like evaluation, migration strategy, prototyping, provisioning, and testing.

#### (a) Cloud Migration Strategies

The type of data and applications the enterprise transfers, and the location are shifted to, significantly impact the migration strategy designed and implemented. There are six main cloud migration strategies—rehosting (lift and shift), re-platforming, repurchasing, refactoring, retiring, and retaining(re-visiting).



## 1. Rehosting (Lift-and-Shift)

Rehosting is the most straightforward and fastest migration path. In this strategy, the application, its data, and its operating system are moved from a physical or virtual on-premise environment directly into a cloud-based virtual machine, such as **Amazon EC2**, with zero changes to the code.

The core philosophy is speed. Because the configuration remains identical, the risk of breaking the application is extremely low. Organizations typically choose this route when facing a hard deadline, such as a data center lease expiration. While it provides a quick exit from physical hardware, the disadvantage is that the application remains "cloud-unaware." It does not automatically scale or heal, and running a non-optimized server in the cloud can sometimes be more expensive than the original setup.

**Pros:** Very fast to implement; requires minimal cloud-specific skills; low risk of application failure.

**Cons:** Does not take advantage of cloud efficiencies; long-term costs can be higher as the app isn't optimized for the cloud.

## 2. Re-platforming (Lift, Reshape, and Shift)

Re-platforming serves as a middle ground between simple rehosting and a complete rewrite. In this strategy, the core architecture of the application remains the same, but specific "heavy" components are swapped out for cloud-managed services to reduce operational overhead.

For example, a company might migrate a web application but decide to move its self-managed database to **Amazon RDS (Relational Database Service)**. This "reshaping" allows the organization to enjoy cloud benefits like automated backups and patching without rewriting the application's business logic. While this requires more testing than a rehost, it significantly reduces the "undifferentiated heavy lifting" of managing infrastructure.

**Pros:** Reduces the "heavy lifting" of managing infrastructure; provides better performance and scaling than simple rehosting.

**Cons:** Requires additional testing to ensure the app is compatible with managed services; potential for "scope creep" during the migration.

## 3. Refactoring / Re-architecting (Cloud-Native)

Refactoring is the most advanced and transformative strategy. It involves a complete redesign of the application to take full advantage of cloud-native features like serverless computing, microservices, and auto-scaling.

This strategy is driven by a need for massive scalability or performance that was impossible in a traditional environment. A monolithic "all-in-one" application is broken down into independent microservices running on **AWS Lambda** or containers. While this is the most expensive and time-consuming route, it offers the highest long-term rewards. The application becomes highly resilient and cost-efficient because it only consumes resources when active.

**Pros:** Maximum scalability and resilience; highly cost-effective as you only pay for resources when they are active.

**Cons:** Very expensive and time-consuming; requires a high level of development expertise; carries a risk of "vendor lock-in."

#### **4. Repurchasing (Drop and Shop)**

The repurchasing strategy involves a total shift in the business model, often moving from a traditional software license to a **Software-as-a-Service (SaaS)** model. Instead of moving code, the organization "drops" the legacy application and "shops" for a modern replacement.

This is most common for standard business utilities like HR management or CRM. By moving to a platform like **Salesforce** or **Microsoft 365**, the company offloads 100% of the technical responsibility to the vendor. The main challenge is data migration—moving historical data from a custom legacy system into a standardized SaaS platform can be complex, and employees will require retraining on the new interface.

**Pros:** Eliminates server and code management entirely; provides instant access to the latest software features.

**Cons:** Moving historical data to a new platform can be complex; requires retraining employees on a new interface.

#### **5. Retiring (Decommissioning)**

During the discovery phase of a migration, IT teams often find applications that are no longer used or provide zero business value. The "Retiring" strategy involves identifying these obsolete assets and shutting them down permanently.

This is a critical step for cost optimization. It prevents the organization from spending money to migrate "ghost" applications that no longer have an active user base. Retiring these apps saves money on cloud storage and strengthens security by reducing the number of potential entry points for hackers.

**Pros:** Instant cost savings on power and licenses; reduces the security "attack surface" of the company.

**Cons:** Requires careful auditing to ensure no critical data is lost; requires business-wide communication.

## 6. Retaining (Re-visiting)

Retaining is a passive strategy where the organization decides to keep certain applications in their current on-premise environment. This decision is usually "re-visited" at a later date.

There are several valid reasons to retain an app: it might be scheduled for an upgrade soon, it may be too complex to move easily, or it may be bound by strict regulatory laws that require data to stay on physical hardware within a specific country. By retaining these sensitive items, the organization can focus its budget on migrating the applications that will provide the most immediate benefit in the cloud.

**Pros:** No immediate risk to critical systems; ensures compliance with data residency laws.

**Cons:** Ongoing hardware maintenance costs; no access to cloud-native innovations for those specific apps.

## Phases of Cloud Migration

There are various ways to go about a cloud migration based on the type of strategy you choose or the size of your organization.

### 1. Evaluation (The Discovery Phase)

Before moving a single server, an organization must perform a 360-degree audit of its current "As-Is" state. This phase is about building the **Business Case**.

**Infrastructure Audit:** Identifying CPU, RAM, and storage usage. Many on-premise servers are "over-provisioned" (e.g., using only 10% of a massive server).

**Dependency Mapping:** Finding out which apps talk to which databases. If you move an app but leave its database behind, the "latency" (lag) will break the system.

**Compliance & Licensing:** Checking if software licenses (like Oracle or Microsoft) can be moved to the cloud or if new ones are needed.

**Example:** A bank evaluates its loan-processing app. They discover it requires a specific hardware security module (HSM) for encryption. They must find out if AWS CloudHSM can replace their physical hardware before proceeding.

---

### 2. Migration Strategy (The Planning Phase)



Once you know *what* you have, you decide *how* to move it. This phase involves choosing between a "Big Bang" approach or a "Phased" approach.

**Hot Plug Strategy (All-at-Once):** Used for independent applications that don't have many "strings attached." You migrate the entire stack (App, DB, Web) in one weekend.

- *Example:* A company's internal holiday-booking portal is moved over a weekend. On Monday, everyone just uses a new URL.

**Fusion Strategy (Hybrid/Partial):** Used for complex systems with deep dependencies. You might move the Web front-end to the cloud but keep the Database on-premise because it runs on a Mainframe that cannot be moved.

- *Example:* A large retail chain moves its website to the cloud to handle "Black Friday" traffic, but the actual "Inventory Management" stays on an old local server because it is physically connected to the warehouse scanners.
- 

### 3. Prototyping (The Proof of Concept)

Never migrate a production environment without a "Test Run." Prototyping (or a Pilot Hole) involves moving a non-critical version of the app to prove the cloud architecture works.

**Validation:** Testing if the cloud network (VPC) and security rules (Firewalls) allow the app to function.

**Performance Benchmarking:** Checking if the app runs as fast in the cloud as it did on-premise.

**Example:** A media company wants to move its video-editing software. In the Prototyping phase, they move one small video-rendering task to the cloud using test data. If the render takes 5 minutes instead of 50, the prototype is a success.

---

### 4. Provisioning (The Implementation Phase)

This is the "Go-Live" preparation. It involves setting up the actual "empty house" in the cloud before moving the "furniture" (data) in.

**Server Provisioning:** Launching EC2 instances, RDS databases, and S3 buckets.

**Optimization:** Right-sizing the servers. If evaluation showed a server only uses 4GB of RAM, you provision a 4GB instance in the cloud, not a 16GB one, to save money.

**Data Replication:** Using tools like **AWS DataSync** or **Database Migration Service (DMS)** to copy live data from the old system to the new cloud system.

**Integration:** Re-pointing the "Web Services" and "API Keys" so the cloud app can talk to the outside world (Payment gateways, Email services, etc.).

**Example:** For a global e-commerce app, the team provisions servers in the Mumbai region (ap-south-1). They set up "Batch Jobs" that automatically back up the database every midnight and configure "Auto-Scaling" so that if users increase, more servers are automatically created.

---

## Approaches for Cloud Migration:

The following are the four broad approaches for cloud migration that have been adopted effectively by vendors:

### 1. Modernizing Existing Applications (The Transformation)

In this approach, a company takes its current software and "re-architects" it to fit the cloud. Instead of just moving the old code, they rebuild parts of it using modern virtualization and container technologies.

**The Strategy:** You don't change everything at once. You improve the app over several small updates (releases) based on what the customers ask for.

**The Challenge:** It requires highly skilled engineers who understand both the "Old" way and the "New" cloud way. It is a slow but steady transformation.

**Example:** A bank takes its 15-year-old desktop software and slowly turns the "Login" and "Balance Check" features into cloud-based mobile services.

---

### 2. Starting from Scratch (The Green Field)

Sometimes, the old software is so outdated that fixing it is a waste of time. In this approach, the company ignores the old app and builds a brand-new "Cloud-Native" version from zero.

**The Strategy:** By starting fresh, the company isn't held back by old mistakes. They can use the newest tools and small, fast teams to build something much better.

**The Benefit:** It avoids confusing old customers with "half-fixed" software and lets engineers focus on the future rather than fixing the past.

**Example:** Instead of trying to make an old DVD-rental software work online, a company builds a brand-new streaming platform like Netflix from scratch.

---

### **3. Creating a Separate Company (The Startup Spin-off)**

Large companies are often slow and afraid of change. This approach involves creating a completely new, independent company (a subsidiary) to handle the cloud business.

**The Strategy:** This new company gets its own brand, its own management, and its own rules. It behaves like a fast-moving "Startup" even though it is owned by a "Giant" parent company.

**The Benefit:** It removes internal politics and "old-school" thinking. The new company can innovate much faster without waiting for the parent company's permission.

**Example:** A traditional taxi company creates a totally separate app-based company with a new name and young staff to compete with Uber.

---

### **4. Buying an Existing Cloud Vendor (The Acquisition)**

If a company has a lot of money but no time, they simply buy a smaller company that is already successful in the cloud.

**The Strategy:** This "Acquisition" does two things: it removes a competitor from the market and instantly gives the big company a working cloud product.

**The Risk:** The biggest danger is that the "Big Company" culture might accidentally destroy the "Small Company" creativity and speed during the merger.

**Example:** Facebook buying **Instagram** or **WhatsApp**. Instead of building their own photo app, they bought the best one in the market to lead the cloud-social space.

---